

Suscripción mensual vía Google Play Store

Propuesta de implementación: alta del producto en Play Console, cobro y verificación del pago, bloqueo/desbloqueo de funcionalidad, y ciclo completo de reactivación tras un pago fallido.

Preparado para: Ezequiel Castiglione

Proyecto: RallyStats (rally_stats)

Fecha: 25 de agosto de 2026 (revisión — incorpora el login con Firebase ya implementado)

Estado: propuesta para discusión — sin cambios aplicados al código

Índice

1. Resumen ejecutivo
2. Contexto y limitaciones actuales del proyecto
3. Alcance de la propuesta
4. Alta del producto de suscripción en Google Play Console
5. Flujo de compra dentro de la app
6. Verificación del pago: por qué no alcanza con confiar en el cliente
7. Arquitectura propuesta
8. Ciclo de vida completo de la suscripción
9. Impacto en la experiencia de usuario
10. Comparativa: RevenueCat vs. backend propio
11. Caso especial: la build de Windows
12. Políticas de Google Play a cumplir
13. Plan de implementación sugerido (por fases)
14. Riesgos y preguntas abiertas para decidir
15. Conclusión y próximos pasos

1. Resumen ejecutivo

Esta propuesta describe cómo agregar una suscripción mensual paga a RallyStats a través de Google Play Billing, de forma que la funcionalidad de la app se bloquee automáticamente si el pago no se procesa, y se vuelva a habilitar automáticamente en cuanto el usuario vuelve a estar al día. Es un documento para discutir decisiones, no un plan de implementación cerrado: hay puntos (sobre todo el de backend propio vs. RevenueCat, y qué hacer con la build de Windows) que conviene definir antes de tocar código.

PUNTO DE PARTIDA IMPORTANTE (ACTUALIZADO)

Desde la primera versión de este informe, el proyecto sumó login con Firebase Authentication + Firestore (para controlar que una cuenta no se use en más de un dispositivo a la vez — ver `lib/services/auth_service.dart`), y funciona tanto en Android como en Windows. Eso significa que la pieza de identidad ("¿quién es este usuario?", un UID estable) que antes había que resolver desde cero para la suscripción **ya existe**. Lo que sigue sin existir es un backend que sepa "¿esta cuenta pagó?" — eso es lo que agrega esta propuesta. Es un cambio de arquitectura más chico que el descrito originalmente, pero sigue siendo uno: los datos de juego (equipos, partidos, pizarra) siguen siendo 100% locales en Hive y no está previsto que dejen de serlo.

Recomendación de alto nivel (desarrollada en la sección 10, y en detalle en el informe complementario "¿Firebase o RevenueCat para administrar las suscripciones?"): usar **RevenueCat** para el cobro/verificación, identificando a cada cuenta con el mismo UID de Firebase que ya usa el login, y sumando la extensión oficial de RevenueCat para Firebase para que el estado de la suscripción también quede reflejado en el mismo proyecto de Firebase (Firestore/Auth) que ya maneja el control de dispositivo único.

2. Contexto y limitaciones actuales del proyecto

Aspecto	Estado actual	Implicancia para la suscripción
Persistencia de datos de juego	100% local con Hive (equipos, partidos, pizarra) — sin cambios respecto a la versión anterior de este informe	Sigue sin haber ningún lugar hoy donde consultar "¿este usuario pagó?" — eso es lo que faltaría agregar
Cuenta de usuario / login	Ya existe — Firebase Authentication (email/contraseña) + Firestore, hoy usado para controlar que una cuenta no se use en más de un dispositivo a la vez (colección <code>account_devices</code>)	Cada cuenta ya tiene un UID de Firebase estable. Es el identificador natural para atar la suscripción a una cuenta en vez de a un dispositivo — ya no hay que introducirlo de cero
Backend	Existe uno mínimo: Firebase Auth + Firestore, dedicado hoy exclusivamente al control de dispositivo único del login	La app ya depende de conectividad y de un backend para el login. Extender ese mismo backend (o RevenueCat, atado a ese UID) para la suscripción es una ampliación, no la incorporación de una pieza totalmente nueva
Distribución y alcance del login	APK (Android) + ejecutable de Windows — el login con Firebase ya funciona en ambas plataformas (en Windows, vía la configuración "web" de Firebase, ver <code>lib/firebase_options.dart</code>)	La identidad de cuenta ya está unificada entre Android y Windows. Lo que sigue sin existir en Windows es el cobro en sí: Google Play Billing sólo funciona en Android instalado desde la Play Store (ver sección 11)
Conectividad	La app sigue diseñada para uso sin conexión (carga en vivo de un partido en una cancha, sin garantía de wifi/datos) — el login ya exige red para iniciar sesión, pero el resto de la app no	La verificación de la suscripción no puede ser "obligatoria en cada apertura" — necesita una caché local con tolerancia offline, igual que se planteaba antes

Ninguno de estos puntos bloquea la propuesta. El cambio más relevante frente a la versión anterior de este informe es que la pieza de **identidad** ya no es parte del problema a resolver — ya existe por el login. Lo que sigue pendiente es exclusivamente la pieza de **verificación de pago** (secciones 6 a 10).

3. Alcance de la propuesta

Esta propuesta cubre específicamente:

- Publicación de un producto de suscripción mensual auto-renovable en Google Play Console.
- Flujo de compra/renovación/cancelación dentro de la app Android.
- Verificación confiable (no falsificable desde el cliente) de que el pago efectivamente se procesó.
- Bloqueo de funcionalidad cuando la suscripción no está activa, y desbloqueo automático cuando vuelve a estarlo.

Explícitamente fuera de alcance de esta primera propuesta (se tratan como preguntas abiertas en la sección 14):

- Qué pasa con la build de Windows (ver sección 11).
- Qué funcionalidad exacta queda bloqueada vs. accesible en modo "lectura" (es una decisión de producto, no técnica).
- Planes anuales, pruebas gratuitas, promociones — se puede agregar después sin cambiar la arquitectura de base.

4. Alta del producto de suscripción en Google Play Console

1. Tener una cuenta de Google Play Console activa (pago único de USD 25) y la app ya subida a la Play Store, al menos en un track de pruebas internas.
2. En **Monetización** → **Productos** → **Suscripciones**, crear un nuevo producto de suscripción:
 - ID del producto (por ejemplo `rallystats_mensual`) — este ID es el que se referencia desde el código, no se puede cambiar después de publicado.
 - Nombre y descripción visibles para el usuario en el checkout de Google.
 - Precio por país/región (Google permite ajustar precios locales o usar conversión automática).
 - Período de facturación: mensual.
 - Período de gracia (grace period): cuántos días sigue Google reintentando el cobro antes de suspender el acceso — configurable, valor típico entre 3 y 7 días.
 - Opcional: período de prueba gratuita, precio promocional de introducción.
3. Configurar las **licencias de prueba** (cuentas de test que pueden "comprar" sin que se cobre de verdad) para poder probar el flujo completo antes de publicar.
4. Activar (si se opta por backend propio o RevenueCat lo requiere) las **Real-time Developer Notifications**: en Play Console se configura un tópico de Google Cloud Pub/Sub al que Google publica cada evento del ciclo de vida de la suscripción.

5. Flujo de compra dentro de la app

Opción	Qué es	Ventajas	Desventajas
<code>in_app_purchase</code> alternativa	Paquete oficial del equipo de Flutter, wrapper directo sobre Play Billing (y StoreKit en iOS, no aplica acá)	Sin dependencia de terceros; control total	La verificación server-side, el manejo de RTDN y la lógica de estados hay que construirlos a mano
<code>purchases_flutter</code> (RevenueCat) recomendada	SDK de terceros que envuelve Play Billing y agrega verificación de servidor y gestión de estado ya resuelta	No hay que programar ni mantener el backend de validación; expone directamente "¿tiene el entitlement activo?"	Dependencia de un tercero; gratis hasta cierto volumen de ingresos, después cobra un % — cambia el modelo de costos del proyecto

En cualquiera de los dos casos, el flujo de compra en la UI es conceptualmente el mismo: el usuario toca "Suscribirse", la app dispara el diálogo nativo de pago de Google Play, y al confirmar, la app recibe una transacción (un `purchaseToken`) que representa esa compra. Ese token es lo que después hay que validar (sección 6).

6. Verificación del pago: por qué no alcanza con confiar en el cliente

El riesgo central de cualquier suscripción in-app es que la app, corriendo en el dispositivo del usuario, no es un entorno confiable. Si la lógica de "¿está pagada la suscripción?" vive únicamente en el cliente (por ejemplo, "si Play Billing me devolvió éxito, marco activo = true"), es falsificable: un APK modificado, un dispositivo rooteado, o simplemente interceptar la respuesta local puede simular una compra que nunca ocurrió ni se cobró. Google Play lo señala explícitamente en su documentación de Billing.

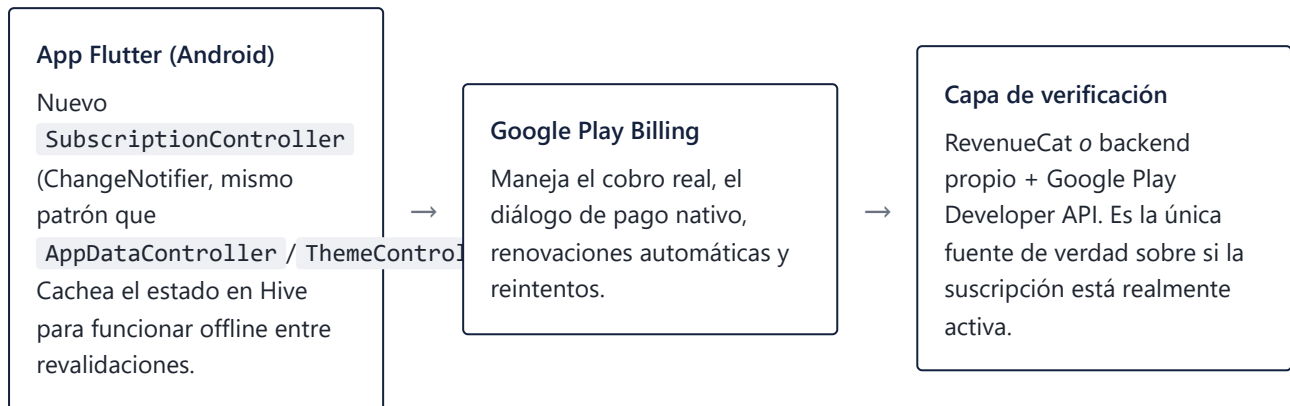
	Verificación client-side	Verificación server-side
Cómo funciona	La app confía en la respuesta que Play Billing le da localmente	El <code>purchaseToken</code> se valida contra un servidor (propio o de RevenueCat) que a su vez consulta a Google
Seguridad	Baja — falsificable	Alta — Google es la única fuente de verdad
Complejidad	Baja	Media/alta si es backend propio, baja si es RevenueCat
Detecta cancelaciones/vencimientos que ocurren fuera de la app	No, o sólo cuando la app vuelve a abrir y reconsulta	Sí, casi en tiempo real vía notificaciones (RTDN)

MECANISMO RECOMENDADO

Validar el `purchaseToken` contra la **Google Play Developer API** (`purchases.subscriptions.get`) desde un servidor, y suscribirse a **Real-time Developer Notifications** (Google publica cada cambio de estado a un tópico de Pub/Sub: renovación, pago fallido, pausa, cancelación, reactivación) para no depender de que la app esté abierta para enterarse.

7. Arquitectura propuesta

Componentes involucrados, de punta a punta:



NOVEDAD RESPECTO A LA VERSIÓN ANTERIOR

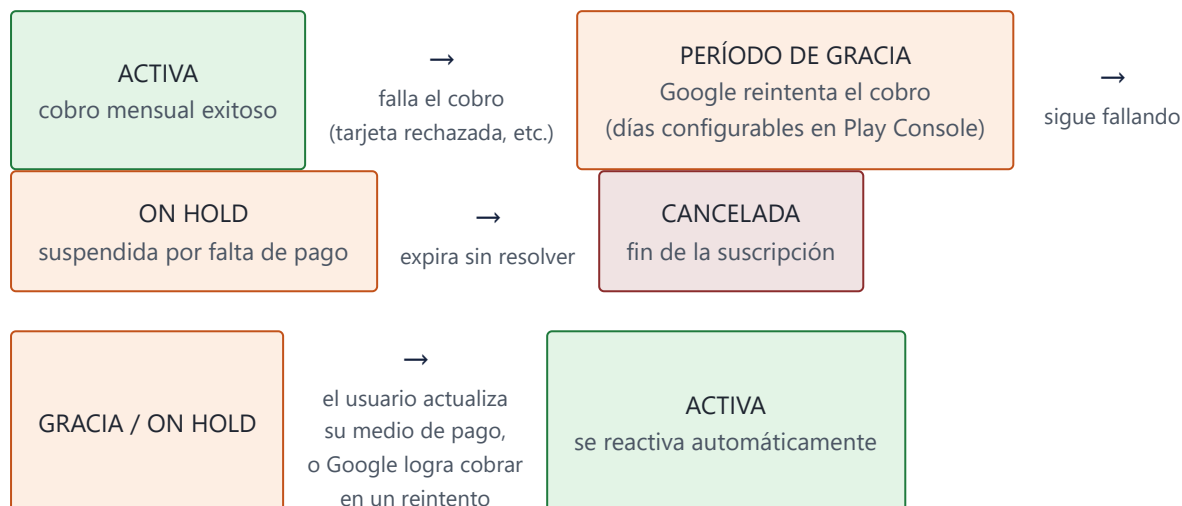
El identificador que ata todo esto a una cuenta ya existe: el UID que Firebase Auth genera al iniciar sesión (el mismo que hoy usa `account_devices` para el control de dispositivo único). Alcanza con pasarlo como App User ID al inicializar RevenueCat (`Purchases.login(firebaseUid)`) para que cada cuenta quede identificada igual en ambos sistemas — no hace falta inventar ni gestionar un identificador nuevo.

El flujo de datos es: la app compra vía Play Billing → el token de esa compra se manda a la capa de verificación (ya asociada al UID de Firebase de la sesión activa) → esa capa confirma contra Google que es válida y está activa → le devuelve a la app un estado de "entitlement activo/inactivo" que el `SubscriptionController` guarda en caché local y expone al resto de la UI.

Cuando el estado cambia del lado de Google (por ejemplo, falla un cobro), la capa de verificación se entera por RTDN sin que la app tenga que estar abierta; la app se pone al día la próxima vez que se abre (o, si se quiere reacción inmediata, con una notificación push — fuera del alcance mínimo de esta propuesta).

8. Ciclo de vida completo de la suscripción

Este es el corazón de la pregunta original: qué pasa exactamente entre que un pago falla y que se vuelve a pagar.



8.1. Qué hace la app en cada transición

Estado en Google	Qué ve/hace la app
Activa	Funcionalidad completa habilitada. Chequeo de rutina en background, sin fricción visible.
Período de gracia	Google sigue considerándola "activa" a efectos de acceso (política recomendada), pero conviene mostrar un aviso no bloqueante ("no pudimos procesar tu pago, actualizá el medio de pago") para que el usuario actúe antes de perder acceso.
On hold	La capa de verificación devuelve "inactiva". El <code>SubscriptionController</code> notifica a la UI y se bloquea la funcionalidad paga, mostrando pantalla de renovación.
Cancelada	Igual que "on hold" en términos de bloqueo; la diferencia es que ya no hay reintentos automáticos de Google — el usuario tiene que volver a suscribirse activamente.
Reactivada (repago)	La capa de verificación se entera (por RTDN o por el próximo chequeo de la app) y el <code>SubscriptionController</code> vuelve a "activa". No hace falta ninguna acción especial del usuario dentro de la app — el desbloqueo es automático en cuanto se revalida el estado.

PUNTO CLAVE

No existe una lógica separada de "reactivación": es la misma consulta de estado que en algún momento anterior devolvió "inactiva" y ahora devuelve "activa". El bloqueo y el desbloqueo son dos caras del mismo mecanismo de revalidación, no dos features distintas.

9. Impacto en la experiencia de usuario

- **Qué se bloquea:** es una decisión de producto a definir, pero lo razonable dado que RallyStats ya tiene datos históricos cargados por el usuario es no ocultar ni borrar esos datos — bloquear funcionalidad "hacia adelante" (cargar un partido nuevo, generar un PDF, exportar) en vez de esconder lo ya cargado.
- **Mensaje al usuario:** pantalla clara de "tu suscripción no está activa" con un botón directo al flujo de renovación (reabre el diálogo de Play Billing o el enlace de gestión de suscripciones de Play Store).
- **Aviso temprano durante el período de gracia:** mostrar un aviso no bloqueante antes de llegar a "on hold" mejora sensiblemente la tasa de recuperación de pagos fallidos (es una práctica estándar recomendada por Google).
- **Botón de "restaurar compras":** buena práctica (y requisito de las políticas de Play) para el caso de reinstalación o cambio de dispositivo con la misma cuenta de Google.

10. Comparativa: RevenueCat vs. backend propio

	RevenueCat recomendado	Backend propio
Infraestructura a mantener	Ninguna — RevenueCat ya tiene el backend de validación	Sí: servidor propio + integración con Google Play Developer API + suscripción a Pub/Sub (RTDN)
Esfuerzo de implementación inicial	Bajo — SDK + configuración en su dashboard	Alto — hay que programar validación, manejo de estados, endpoints, persistencia de eventos
Costo económico	Gratis hasta cierto volumen de ingresos mensuales (revisar el umbral vigente al momento de decidir), después cobra un porcentaje de los ingresos	Sólo el costo de hosting propio (potencialmente más barato a escala, pero con costo de desarrollo/mantenimiento)
Control y dependencia de terceros	Depende de la disponibilidad y las políticas de un servicio externo	Control total
Encaje con el proyecto actual	Alto — no hay que programar ni mantener la lógica de validación de pagos, la parte de mayor riesgo	Medio — a diferencia de la versión anterior de este informe, ya no es "adoptar un backend de cero": el proyecto ya tiene Firebase Auth + Firestore corriendo por el login. Sigue siendo trabajo real (webhooks de RTDN, llamadas a la Google Play Developer API, manejo de todos los estados), pero se apoya en infraestructura que ya existe en vez de sumar una plataforma nueva

ACTUALIZACIÓN TRAS SUMAR EL LOGIN CON FIREBASE

Con el login ya resuelto sobre Firebase, la disyuntiva "RevenueCat vs. backend propio" se volvió más pareja de lo que este informe planteaba originalmente — se analiza en profundidad, con una tercera alternativa híbrida, en el informe complementario "**¿Firebase o RevenueCat para administrar las suscripciones?**". Su conclusión, que actualiza la recomendación de esta sección: usar **RevenueCat** para el cobro y la validación (sigue siendo el motor más confiable y con menos mantenimiento propio) sumando su **extensión oficial de Firebase**, que refleja automáticamente el estado de la suscripción en el mismo proyecto de Firestore/ Firebase Auth que ya maneja el login — evitando así tener que mirar dos paneles separados para administrar una cuenta.

En síntesis, se mantiene **RevenueCat** como recomendación para la validación de pagos en sí (resuelve la parte más riesgosa sin código propio sin probar en producción), pero ya no como una

plataforma aislada del resto del proyecto: gracias a su extensión de Firebase, su estado queda integrado al mismo lugar donde ya vive el login y el control de dispositivo único.

11. Caso especial: la build de Windows

Google Play Billing **no existe** fuera de Android instalado vía Play Store. El ejecutable de Windows no tiene ningún mecanismo nativo para validar esta suscripción. Lo que sí cambió desde la versión anterior de este informe: el login con Firebase ya funciona igual en Android y en Windows (ver sección 2), así que la identidad de cuenta ya está unificada entre ambas plataformas — el problema pendiente es exclusivamente el **cobro** en Windows, no la identificación del usuario. Opciones, para definir en conjunto (ninguna es técnicamente "la correcta", es una decisión de producto/negocio):

1. **Dejar la build de Windows fuera del esquema de suscripción** (gratis o con el modelo que tenga hoy) — la opción más simple, pero dos usuarios con el mismo caso de uso pagarían distinto según la plataforma.
2. **Extender la verificación a Windows contra la misma cuenta de Firebase:** como la identidad ya es la misma en ambas plataformas, esto es más viable ahora que antes de sumar el login — pero el cobro en sí igual tendría que hacerse por otro medio (Stripe, Mercado Pago, licencia manual, etc.), ya que no hay Play Billing en desktop. Sigue siendo una integración de cobro aparte, aunque ya no una identidad aparte.
3. **No ofrecer más la build de Windows**, o mantenerla como versión gratuita/limitada distinta de la de Android.

Esta decisión conviene tomarla antes de implementar, porque afecta si vale la pena diseñar el `SubscriptionController` pensando en múltiples fuentes de verificación desde el principio, o si alcanza con acoplarlo directamente a Play Billing.

12. Políticas de Google Play a cumplir

- Mostrar claramente el precio, la periodicidad y que es auto-renovable *antes* de que el usuario confirme la compra.
- Ofrecer una forma de cancelar y de restaurar compras accesible desde la app (o al menos un enlace directo a la gestión de suscripciones de Play Store).
- No usar patrones de diseño engañosos ("dark patterns") para dificultar la cancelación.
- Usar cuentas de licencia de prueba durante el desarrollo — Google puede rechazar builds que no permitan probar el flujo de compra correctamente en revisión.
- Si se bloquea contenido ya generado por el usuario (no sólo funcionalidad nueva), Google es más estricto en la revisión — otro argumento para el enfoque de la sección 9 (bloquear hacia adelante, no esconder datos históricos).

13. Plan de implementación sugerido (por fases)

Fase	Contenido
0. Decisiones previas	Resolver las preguntas de la sección 14 (RevenueCat vs. propio, qué se bloquea, qué pasa con Windows) antes de escribir código.
1. Alta en Play Console	Producto de suscripción, licencias de prueba, (si aplica) configuración de RTDN.
2. Integración del SDK de compra	RevenueCat o <code>in_app_purchase</code> ; flujo de compra end-to-end probado con cuentas de test.
3. <code>SubscriptionController</code> + caché local	Estado de suscripción como <code>ChangeNotifier</code> , persistencia en Hive, revalidación al abrir la app.
4. Bloqueo de funcionalidad en la UI	Pantallas/acciones envueltas por el estado de suscripción, pantalla de renovación, botón de restaurar compras.
5. Pruebas del ciclo completo	Simular con cuentas de test: compra → cancelación manual → verificar bloqueo → reactivar → verificar desbloqueo automático.
6. Documentación y manual de usuario	Actualizar <code>tool/generate_manual.dart</code> y el README, según el proceso de release ya establecido en el proyecto.

14. Riesgos y preguntas abiertas para decidir

¿RevenueCat (+ su extensión de Firebase) o backend propio sobre Firebase?

Impacta directamente el esfuerzo de implementación y el modelo de costos a futuro. Ver el informe complementario "¿Firebase o RevenueCat para administrar las suscripciones?" para el detalle completo — la recomendación actual es RevenueCat + su extensión oficial de Firebase (sección 10).

¿Qué pasa con los usuarios que ya tienen la app instalada hoy, sin suscripción?

¿Se les da un período de gracia/versión gratuita "legacy", o la suscripción aplica sólo a instalaciones nuevas?

¿Qué funcionalidad exacta queda bloqueada?

¿Toda la carga en vivo, sólo la generación de PDF, sólo la exportación? Definir esto antes de tocar la UI.

¿Qué hacemos con la build de Windows?

Ver sección 11 — no tiene una solución técnica única, es una decisión de negocio.

¿Hace falta reacción inmediata (push) al vencer/reactivar, o alcanza con revalidar al abrir la app?

Revalidar al abrir es mucho más simple; push en tiempo real agrega otra pieza de infraestructura (notificaciones).

¿Se ofrece algún período de gracia propio de la app (además del de Google) antes de bloquear?

Por ejemplo, no bloquear hasta 24-48h después de "on hold" para evitar bloqueos en medio de un partido por un problema transitorio de conexión.

15. Conclusión y próximos pasos

Técnicamente el mecanismo es conocido y bien soportado por Google Play y por Flutter; el trabajo real está en dos lugares: (1) elegir entre RevenueCat y backend propio, y (2) decidir el alcance exacto del bloqueo y qué hacer con la build de Windows. Ninguna de las dos decisiones requiere escribir código todavía — conviene cerrarlas primero.

Próximo paso sugerido: revisar juntos la sección 14 y, con esas respuestas, recién ahí armar un plan de implementación concreto (issues, orden de trabajo, estimación de tiempo).